Eigen-Séta: performance and security white paper

Measured cost of every operation at every parameter set, security levels, known attacks, risks and open questions

Anonymous

2026-10-17 — research document, not peer reviewed

Abstract

Eigen-Séta v2 is an isogeny-based key encapsulation mechanism whose trapdoor is a secret endomorphism of the public curve. This white paper gives the measured cost of key generation, encapsulation, decapsulation and authenticated public-key encryption on all five parameter sets of the maintained Rust implementation, all measured on one host in one campaign. It then states the security level of each set in the two metrics that matter for NIST's categories, catalogues every attack we know, and ranks the risks and open questions. The specification, proofs and full analysis are in the companion paper [Ano26a]. The short version is as follows.

- Public keys and ciphertexts are 1.3–5.2 kB. A shared secret is 32 bytes.
- On a shared 4-vCPU cloud VM, L1 decapsulates in about 1.2 s and encapsulates in 0.20 s. Key generation is a rejection search, from seconds at L1 to 14 min at Q3. Q5 key generation took 33 min with four workers.
- Against every NIST post-quantum KEM measured on the same host, Eigen-Séta is slower by orders of magnitude. At level 1 it decapsulates about 89,000 times slower than ML-KEM and 2,600–7,600 times slower than BIKE, FrodoKEM and Classic McEliece. Its 1.3 kB keys and ciphertexts are competitive.
- L1, L3 and L5 meet NIST categories 1, 3 and 5 only in NIST's gate count under MAXDEPTH. At least 2^{λ} quantum queries is met only by L5 (at $\lambda = 128$), Q3 and Q5.
- The largest risk is the hidden orientation of the public curve. If it became effective, it would give subexponential quantum key recovery at every set.

Security is a conjecture. No one outside the project has analysed the scheme. Do not use it to protect data.

Contents

1 Summary **2**

2 The scheme in one page **2**

3 Parameter sets and sizes **3**

4 Performance **4**
4.1 Method 4
4.2 Results 4

5 How far off: comparison with other post-quantum KEMs **6**
5.1 Same host 6
5.2 Other isogeny schemes (published figures) 7
5.3 What would close the gap 7

6 Security level **8**
6.1 How the levels are computed 8
6.2 Levels per set 9
6.3 What this means for NIST categories 9

7 Attacks **10**
7.1 Key recovery 10
7.2 Message recovery 10
7.3 Chosen-ciphertext, implementation and physical attacks 11

8 Risks, big unknowns and open questions **11**
8.1 Open questions 12

9 Implementation status **12**

10 Reproduction **12**

1 Summary

Table 1 is the one-page answer. Every number is measured in run `run-20261017T2022Z-kem-bench` (§4) or derived by the cost model (§6). The record of each value is named in the section that discusses it.

Table 1: The five parameter sets at a glance. Sizes are exact. Times are medians on one shared 4-vCPU VM (§4.1). Security columns are $\log_2$ of the cheapest known attack, from the cost model.

set	NIST cat. (gates)	$\geq 2^\lambda$ queries	$\log_2 p$	pk = ct (B)	sk (B)	keygen (s)	encaps (s)	decaps (s)	classical	quantum queries
L1	1	no	1341	1344	1855	5	0.20	1.20	128.4	64.5
L3	3	no	1991	1992	2730	84	1.14	5.83	192.6	96.5
L5	5	yes (λ=128)	2630	2632	3594	51	2.30	12.78	256.8	128.5
Q3	3	yes	3904	3904	5311	815	5.88	37.34	384.4	192.5
Q5	5	yes	5192	5192	7050	2005	13.70	88.66	512.7	256.5

Recommended use, if the conjectures hold: L5, Q3 and Q5 for post-quantum deployments; L1 and L3 only where security against classical attack and NIST's gate-count metric are enough.

2 The scheme in one page

Let E_D be the curve with complex multiplication by $\mathbb{Z}[(1+\sqrt{-7})/2]$ ($j=-3375$) over $\mathbb{F}_p$, with $p = 2^{E+1}3^F c - 1$. Write ω for its endomorphism of degree 2, and (P_D, Q_D) for an eigenbasis of ω on $E_D[2^E]$.

Key generation. Draw a secret odd degree f from an interval of about $2^{2\lambda+20}$ values. Find $\alpha \in \mathrm{End}(E_D)$ of norm $f(2^E - f)$ with Cornacchia. Split the Kani isogeny with kernel $\{([f]R, \alpha R)\}$ to obtain $\varepsilon : E_D \to E_A$ of degree f. The public key is E_A with x-coordinates of $\lambda_1\varepsilon(P_D)$, $\lambda_2\varepsilon(Q_D)$ and their difference. These points are eigenvectors of the hidden endomorphism $\tau = \varepsilon\omega\hat{\varepsilon}$. The secret key is a function of f alone.

Encapsulation. Pick s modulo $d = 3^F$. Walk the 3^F-isogeny φ with kernel $P_3 + sQ_3$ to E_C. Push the public basis through it and mask the images by $(a, b) = G(s, H(pk))$. The ciphertext is E_C and three x-coordinates. The key is $H_K(s, ct)$, under the Fujisaki–Okamoto transform with implicit rejection.

Decapsulation. The masks act diagonally on eigenvectors, so the holder of f knows the action of the trace-zero endomorphism $\psi = 2\varphi\tau\hat{\varphi} - tdf$ on the masked points. One $(2^E, 2^E)$-isogeny in dimension two (Kani's lemma, theta coordinates) evaluates ψ on $E_C[3^F]$. Its kernel gives $\hat{\varphi}$ and hence s. The decapsulator then re-encrypts and compares.

Why it is secure, if it is. Without f, the public key is an isogeny of unknown degree f from a known curve, and the ciphertext is a FESTA-type masked-torsion instance [BMP23]. The Castryck–Vercauteren observation [CV23], that an eigenbasis of a *known* endomorphism defeats diagonal masks, is the trapdoor. Hiding the endomorphism's degree is what is supposed to deny it to everyone else.

Figure 1 shows the objects and the attack routes of §7.

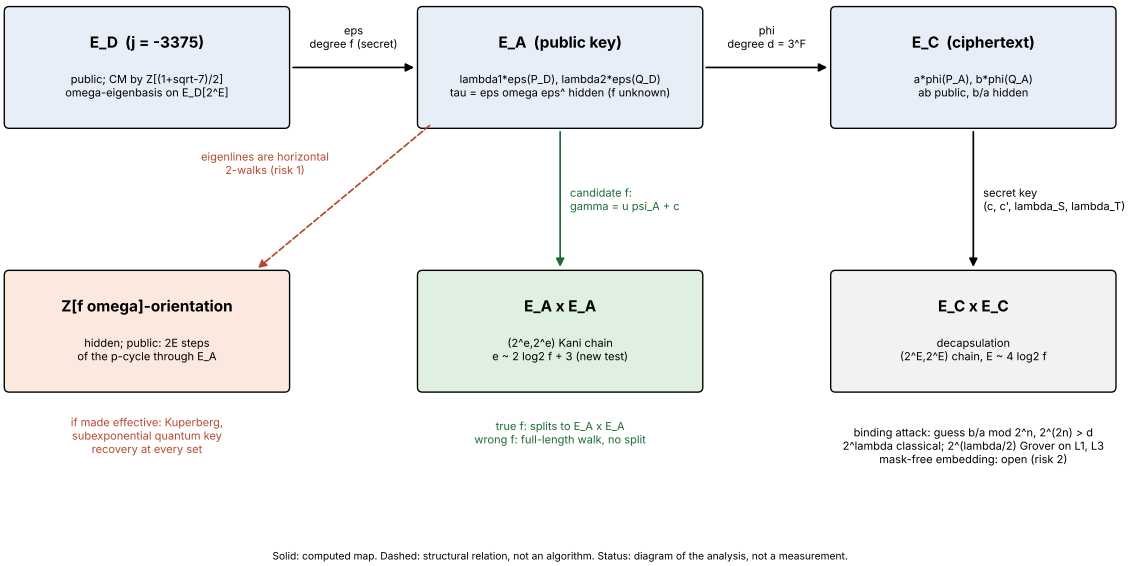


Figure 1: Objects of the scheme and the attack routes considered. Diagram of the analysis, not a measurement.

3 Parameter sets and sizes

The primes come from a deterministic public search [Ano26b]. Fix λ and set $m = 2$ for the L sets or $m = 4$ for the Q sets. Then $F = \lceil m\lambda / \log_2 3 \rceil$, E is the smallest exponent with $2^E > 28d^2 f_{\max}^2$, and c is the smallest cofactor for which p is prime under fixed-base Miller–Rabin and $p \equiv 3, 5, 6 \pmod 7$. At $\lambda = 128$ and $m = 4$ the search reproduces the L5 prime, so L5 is also the quantum-hardened level-1 set.

The prime grows as $p \approx d^5$, about 10.5λ bits for the L sets. The reason is that one Kani chain must decode an endomorphism of degree $7d^2 f^2$. Two half-chains would cut this to about d^3 (§8.1). That is the single largest efficiency lever.

Table 2: Parameters and object sizes in bytes. Public key and ciphertext are four $\mathbb{F}_{p^2}$ elements each. Files add an 8-byte header. A PKE ciphertext is the KEM ciphertext, a 12-byte nonce, the message and a 16-byte tag. Sizes measured by the benchmark (`sizes` rows); parameters from `eigenx:src/params_generated.rs`.

set	λ	$\log_2 p$	limbs	E	F	$\log_2 d$	c	f bits	pk = ct	sk	PKE overhead
L1	128	1341	21	1076	162	256.8	144	277	1344	1855	1372
L3	192	1991	32	1594	243	385.1	1059	409	1992	2730	2020
L5	256	2630	42	2106	324	513.5	380	536	2632	3594	2660
Q3	192	3904	61	3125	485	768.7	371	791	3904	5311	3932
Q5	256	5192	82	4155	647	1025.5	975	1049	5192	7050	5220

4 Performance

4.1 Method

One campaign, run `run-20261017T2022Z-kem-bench`, measured every operation on one host: a shared cloud VM with 4 vCPUs (Intel Xeon at 2.10 GHz, one thread per core), Linux 6.18, rustc 1.97.0, release profile, `eigenx` commit `d89d790`. Isolation is the lowest level (L0): no core pinning for the all-core rows, and `taskset -c 0` for the single-core rows. Other tenants' load is not controlled. The harness is `eigenx:examples/bench_kem.rs`.

- For each set it draws several fresh keys. Key generation is timed in wall time and in process CPU time, because it is a rejection search: a single sample says little, and parallel attempts (`jobs`) trade CPU time for wall time.
- Under each key it runs encapsulation then decapsulation of an honest ciphertext several times, one PKE round trip of a 1 KiB message with associated data, and one public-key degree test, the attacker's oracle (§6). Every result is checked.
- In the same binary it times $\mathbb{F}_p$ and $\mathbb{F}_{p^2}$ arithmetic.

Samples per set: L1 5 keys × 10, L3 and L5 5 × 5, Q3 2 × 3, Q5 1 × 3, and single-core L1 2 × 5, L3 and L5 1 × 3. The Q sets use `jobs`=4 for key generation; every other row uses `jobs`=1. Medians are reported with the range, because the samples are few and the host is noisy.

4.2 Results

Table 3: Wall time in seconds, median [min–max] over n samples, all four vCPUs available (the KEM uses scoped threads inside decapsulation and encapsulation). Key generation is one attempt loop per key (`jobs`=1 for L, `jobs`=4 for Q). Every decapsulation and PKE decryption returned the right value. Run `run-20261017T2022Z-kem-bench`.

set	keygen wall	keygen CPU	encaps	decaps	PKE enc / dec (1 KiB)
L1 (jobs=1)	4.6 [3.6–6.5], n=5	5.4 [4.5–7.5], n=5	0.204 [0.165–0.304], n=50	1.20 [1.08–1.80], n=50	0.208 / 1.25
L3 (jobs=1)	84.1 [76.5–94.0], n=5	87.8 [79.8–97.3], n=5	1.136 [0.796–1.574], n=25	5.83 [5.40–6.30], n=25	1.038 / 6.24
L5 (jobs=1)	50.6 [39.2–80.3], n=5	60.2 [48.6–89.2], n=5	2.301 [1.744–3.063], n=25	12.78 [11.81–14.90], n=25	2.281 / 13.08
Q3 (jobs=4)	815.3 [689.9–940.6], n=2	2592.3 [2395.0–2789.7], n=2	5.880 [5.381–6.562], n=6	37.34 [36.12–38.49], n=6	5.929 / 37.13
Q5 (jobs=4)	2004.7, n=1	6908.6, n=1	13.695 [13.478–14.137], n=3	88.66 [87.77–89.67], n=3	13.301 / 89.01

Reading the numbers.

- *Decapsulation dominates online cost.* It is a $(2^E, 2^E)$ theta chain plus pairings and a re-encryption. At L1 it is about six times the cost of encapsulation, a 3-isogeny chain of length F. Across sets it scales roughly as $E \cdot M(p)$, where $M(p)$ is the $\mathbb{F}_{p^2}$ multiplication time of Table 5.

Table 4: The same operations restricted to one core (`taskset -c 0`). Median [min–max]. Run `run-20261017T2022Z-kem-bench`.

set	keygen	encaps	decaps
L1	6.5 [4.6–8.5], n=2	0.240 [0.202–0.341], n=10	1.88 [1.78–2.12], n=10
L3	85.2, n=1	1.055 [1.044–1.086], n=3	9.61 [9.33–9.65], n=3
L5	135.7, n=1	2.938 [2.879–2.992], n=3	22.20 [21.51–25.18], n=3

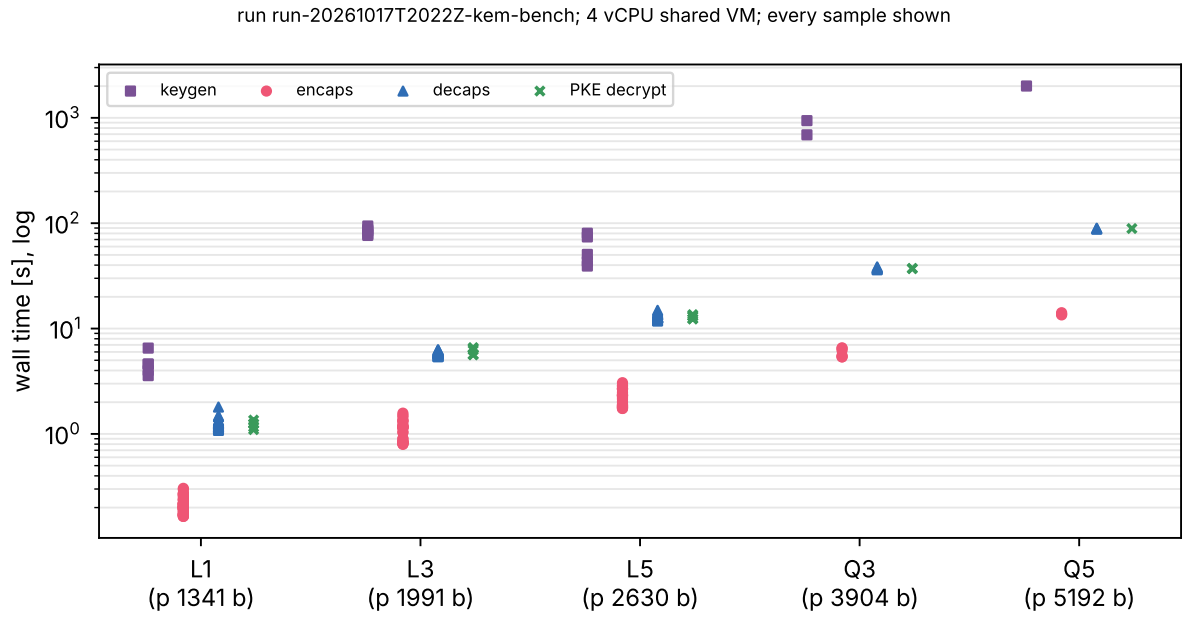


Figure 2: Every sample of every operation, log scale, all cores. Measured; run `run-20261017T2022Z-kem-bench`.

- *Threads.* Decapsulation runs four independent ladders and the chain's two Montgomery conversions in parallel. On one core it is about 1.6 times slower at L1 and 1.7 times at L3 and L5 (Table 4). Encapsulation barely changes.
- *Key generation is a rejection search.* Its time per key varies by up to a factor of two within a set, so medians over five keys are rough. With `jobs`=4, Q3 used 2.4–2.8 ks of CPU per key, finished in 11.5–15.7 minutes of wall time, and used the four cores at about 75%. L5 key generation is faster than L3 at the median. The rejection rate, not the field size, sets the cost, and five samples cannot resolve the difference.
- *PKE* adds one ChaCha20-Poly1305 pass to the KEM. For a 1 KiB message it is indistinguishable from the KEM times.
- *Comparison with the historical build.* Revision 1 of the companion paper measured decapsulation with a Mumford-coordinate chain: 13.4–15.0 s at L1, 51.8–53.9 s at L3 and 120–124 s at L5, on a 2.80 GHz VM. The theta build is roughly 10 times faster at every L set. Those are different hosts, so treat this as an order of magnitude, not a measured speedup.
- *No external comparison.* No other implementation of this scheme exists, so there is no external tool to compare against.

Table 5: Field arithmetic in the same binary: $\mathbb{F}_p$ and $\mathbb{F}_{p^2}$ multiplication (best of 15 loops of 20,000) and $\mathbb{F}_p$ inversion (mean of 200). Measured, ns.

set	limbs	$\mathbb{F}_p$ mul	$\mathbb{F}_{p^2}$ mul	$\mathbb{F}_p$ inversion
L1	21	536	1674	36,539
L3	32	1760	5479	59,891
L5	42	2957	9019	88,919
Q3	61	5936	18343	152,385
Q5	82	10656	32345	240,125

5 How far off: comparison with other post-quantum KEMs

5.1 Same host

Table 6 runs every post-quantum KEM that NIST standardised or shortlisted on the benchmark host, through liboqs 0.13.0 (the `oqs` Rust crate 0.11.0). Each operation is timed individually on one pinned core, until 2 s or 1,000 operations. The table shows the mean of the medians of two repetitions. Record: `crypto:research/eigen_seta_whitepaper_20261017/records/pqkem_same_host/`. liboqs was built as a distribution build, so it picks its AVX2 or AVX-512 code at run time on this CPU. The one exception is HQC: liboqs ships only PQClean's portable implementation for it, so the HQC rows understate an optimised HQC. The Eigen-Séta rows are the all-core medians of Table 3. An earlier measurement of ML-KEM-512 with the portable RustCrypto crate (no AVX2), taken while Q5 key generation loaded the host, gave 37 and 47 μs for encapsulation and decapsulation (`records/mlkem_same_host/`). It is kept as a record but not used in the table. Eigen-Séta's decapsulation uses up to four threads, while liboqs uses one, which flatters Eigen-Séta by up to about 1.7 times (Table 4).

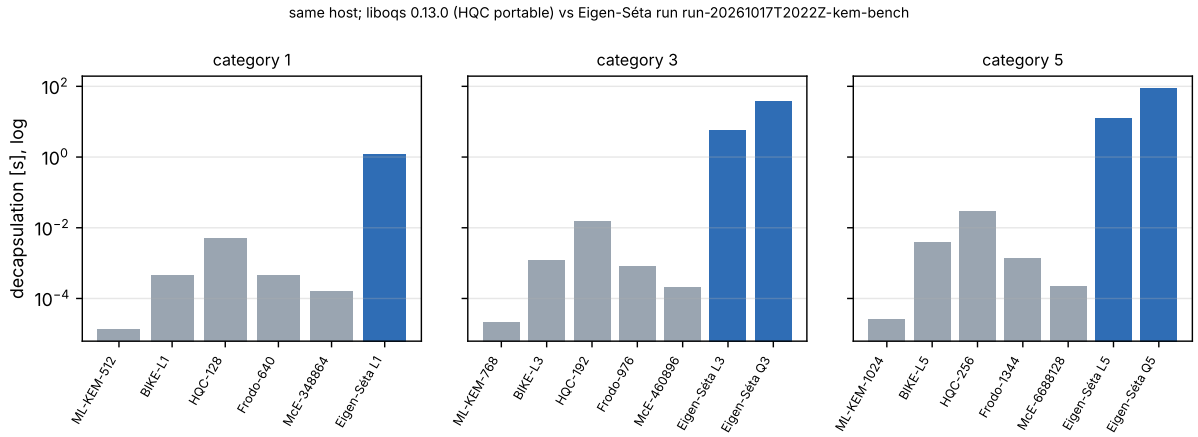


Figure 3: Decapsulation time on the benchmark host, log scale, by NIST category. Grey: liboqs 0.13.0 (HQC portable). Blue: Eigen-Séta. Measured.

What the table says.

- *Against ML-KEM, the standard,* Eigen-Séta L1 is about 19,000 times slower to encapsulate and 89,000 times slower to decapsulate. By set, the decapsulation ratio is L1 89,000, L3 280,000, L5 510,000, Q3 1,800,000, Q5 3,500,000. That runs from four orders of magnitude at L1 encapsulation to six and a half at Q5 decapsulation.
- *Against the code-based and conservative lattice KEMs:* at level 1, BIKE, FrodoKEM and Classic McEliece decapsulate 2,600 to 7,600 times faster than Eigen-Séta L1. Even the

Table 6: Post-quantum KEMs on one host. Median wall time per operation; sizes in bytes (k = thousand). Every decapsulation was checked. †Portable reference implementation (no AVX2).

KEM	keygen	encaps	decaps	pk	ct
Category 1					
ML-KEM-512	10 µs	11 µs	14 µs	800	768
BIKE-L1	263 µs	39 µs	458 µs	1,541	1,573
HQC-128†	1.7 ms	3.3 ms	5.1 ms	2,249	4,433
FrodoKEM-640-AES	320 µs	458 µs	441 µs	9,616	9,720
McEliece-348864	40.7 ms	14 µs	158 µs	261 k	96
Eigen-Séta L1	4.59 s	203.7 ms	1.20 s	1,344	1,344
Category 3					
ML-KEM-768	17 µs	17 µs	21 µs	1,184	1,088
BIKE-L3	531 µs	88 µs	1.2 ms	3,083	3,115
HQC-192†	5.1 ms	10.4 ms	15.5 ms	4,522	8,978
FrodoKEM-976-AES	621 µs	926 µs	825 µs	15,632	15,744
McEliece-460896	124.4 ms	29 µs	208 µs	524 k	156
Eigen-Séta L3	84.13 s	1.14 s	5.83 s	1,992	1,992
Eigen-Séta Q3	815.27 s	5.88 s	37.34 s	3,904	3,904
Category 5					
ML-KEM-1024	19 µs	20 µs	25 µs	1,568	1,568
BIKE-L5	1.5 ms	177 µs	3.8 ms	5,122	5,154
HQC-256†	9.5 ms	19.3 ms	28.9 ms	7,245	14,421
FrodoKEM-1344-AES	1.1 ms	1.5 ms	1.4 ms	21,520	21,632
McEliece-6688128	261.4 ms	50 µs	216 µs	1045 k	208
Eigen-Séta L5	50.57 s	2.30 s	12.78 s	2,632	2,632
Eigen-Séta Q5	2004.73 s	13.70 s	88.66 s	5,192	5,192

portable HQC is about 230 times faster. Classic McEliece's key generation, the slowest in the field at 40 ms to 0.3 s, is still 100 to 200 times faster than ours.

- *Where Eigen-Séta is competitive: size.* At level 1 its 1,344-byte key and ciphertext are close to ML-KEM and BIKE, smaller than HQC and FrodoKEM, and 190 times smaller than a Classic McEliece public key. The quantum-hardened sets lose much of that edge (3.9–5.2 kB).

5.2 Other isogeny schemes (published figures)

No other isogeny encryption scheme is available in a library we could run. Their published figures are on other hardware: PIKE in C on an Intel i7 at 2.3 GHz encrypts in 23 ms and decrypts in 15 ms at NIST level I [PIK26]. POKÉ's SageMath proof of concept takes about 100 ms for each, with a 431-bit prime, a 324-byte public key and a 648-byte ciphertext [BM24]. FESTA's SageMath implementation on an Apple M1 Pro takes 3.09 s to encrypt and 9.14 s to decrypt, with a 1292-bit prime and 561 and 1,122-byte key and ciphertext [BMP23]. One constant-time CSIDH-512 group action is about 125 Mcycles on Skylake [BBC+21]. Against PIKE, Eigen-Séta L1 decrypts about 80 times slower and encrypts about 9 times slower. The main reason is the prime: about $10\lambda/3$ bits for POKÉ and PIKE against 10.5λ here.

5.3 What would close the gap

Two half-chains would bring the prime to about d^3, roughly 800 bits at L1 (§8.1). Field arithmetic was portable Rust and can be improved (it since has been: aburan28/eigenx#6). The theta step

itself is already at the cost of its formulas, so the chain gains little from step-level work. Our estimate for all of this together is about one order of magnitude. That would put Eigen-Séta level with POKÉ and PIKE, and still three to four orders of magnitude behind ML-KEM and the code-based KEMs at decapsulation.

6 Security level

6.1 How the levels are computed

The levels are *derived*, not measured: no attack in Table 8 can be run at real size. They come from the cost model in `eigenx:src/security.rs`, run by `eigenx:examples/security_estimate.rs` (record `crypto:research/eigen_seta_whitepaper_20261017/records/security_estimate.txt`). For each attack the model gives three numbers: classical operations, quantum oracle queries, and quantum gates under NIST's MAXDEPTH. The gate count uses the parallel-Grover accounting: a search over 2^n items with an oracle of 2^g gates and depth 2^{g-4} costs $2^{n/2+g}$ gates if serial Grover fits the depth limit, and $2^{n+(g-4)+g-\mathrm{MAXDEPTH}}$ otherwise. The model favours the attacker in that it ignores memory and uses $p^{1/3}$ for path finding. The tables in this section charge 2^8 $\mathbb{F}_{p^2}$ multiplications of Karatsuba cost per $(2,2)$-step, which the next paragraph corrects to $2^{7.5}$.

What we measured, and a correction. One input of the model can be checked: the cost of an isogeny-chain step, the Grover oracle's inner loop. Table 7 times the attacker's public-key degree test and divides by its length e and by the $\mathbb{F}_{p^2}$ multiplication time, giving 583–720 "multiplication-equivalents" per step. That figure is not a step cost. It includes the work before the chain (about half of the test, mostly ladders) and every field addition. An earlier version of this section read it as showing that the model's 2^8 errs on the attacker's side. That reading was wrong.

Exact operation counts settle it (`eigenx:examples/chain_opcount.rs`, feature `opcount`; aburan28/eigenx#7). A theta $(2,2)$-step of the same test costs 566.6 $\mathbb{F}_p$ products, which is 188.9 $\mathbb{F}_{p^2}$ multiplications, or $2^{7.56}$. The model's 2^8 therefore overcharged the attacker. eigenx#7 lowers the constant to $2^{7.5}$, the measured cost rounded down. That lowers every quantum-gate column of the mask-guess and degree-search rows by 1.0 bit at MAXDEPTH 2^{40} and 2^{64}, and by 0.5 bit at 2^{96}. The classical and quantum-query floors are unchanged. Table 8 and Fig. 4 still show the 2^8 model; with the correction the L1 gate floor is $2^{150.2}/2^{126.2}/2^{97.1}$ against AES-128's $2^{130}/2^{106}/2^{74}$. Tier A still holds for L1, L3 and L5, with margins of about 20 bits.

Table 7: Measured cost of the attacker's chain step: one public-key degree test of length e for the true f of each key (accepted in every case). "Mult.-equiv." is wall time per step divided by $\mathbb{F}_{p^2}$ multiplication time; it includes setup and additions, so it is not a step cost (see text). Median over keys. Run `run-20261017T2022Z-kem-bench`.

set	e	time per test (s)	μs per step	mult.-equiv. per step
L1	569	0.63	1122	670
L3	828	2.99	3640	664
L5	1091	6.45	5922	657
Q3	1598	18.69	11697	638
Q5	2139	42.79	20004	618

Table 8: Cost of each attack, $\log_2$ (derived). Chains for the degree search are the public-key test of §7. The hidden-orientation row is a lower bound under the false assumption that the class-group action can be evaluated, not an attack.

target	attack	L1	L3	L5	Q3	Q5
key	degree search, classical (chains)	265.5	396.9	523.5	777.9	1035.5
key	degree search, Grover (queries)	132.7	198.4	261.7	389.0	517.8
key	$\mathrm{End}(E_A)$, classical $p^{1/3}$	447.0	663.7	876.7	1301.3	1730.7
key	$\mathrm{End}(E_A)$, quantum $p^{1/4}$	335.2	497.8	657.5	976.0	1298.0
key	hidden orientation, $f^{1/2}$ (lower bound)	138.5	204.5	268.0	395.5	524.5
message	mask guess + Kani, classical	129.0	193.0	257.0	385.0	513.0
message	mask guess + Kani, Grover (queries)	64.5	96.5	128.5	192.5	256.5
message	claw on φ, classical $\sqrt{d}$	128.4	192.6	256.8	384.4	512.7
message	claw on φ, quantum $d^{1/3}$	85.6	128.4	171.2	256.2	341.8
classical floor		128.4	192.6	256.8	384.4	512.7
quantum-query floor		64.5	96.5	128.5	192.5	256.5
gate floor, MAXDEPTH $2^{40}/2^{64}/2^{96}$		151/127/98	218/194/162	284/260/228	415/391/359	545/521/489
AES reference (NIST)		130/106/74	193/169/137	258/234/202	193/169/137	258/234/202

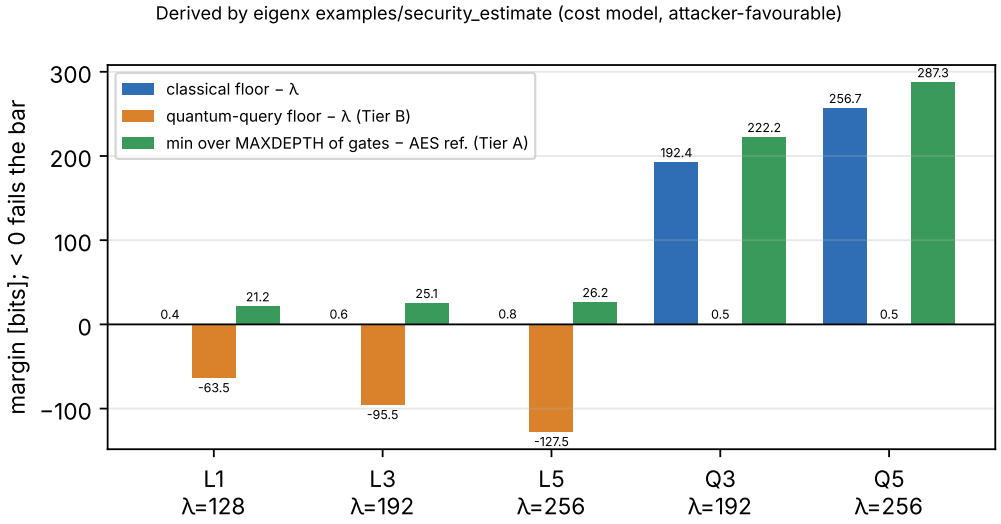


Figure 4: Margin of each set over its target in each metric (derived). A negative bar fails that bar.

6.2 Levels per set

6.3 What this means for NIST categories

Tier A: gate count under MAXDEPTH (NIST's stated metric).
L1, L3 and L5 clear the AES-128, -192 and -256 references at all three depth limits. The margins are 21, 25 and 26 bits at the tightest limit. In this metric they are *conjectured* to be categories 1, 3 and 5.

Tier B: at least 2^λ quantum queries against every known attack.
The binding attack on the L sets, Grover over the mask ratio, needs only $2^{\lambda/2}$ queries. That is $2^{64.5}$ at L1 and $2^{96.5}$ at L3, the same exposure as FESTA. Only L5 (at $\lambda = 128$), Q3 and Q5 meet this bar. Recommending L1 or L3 as quantum-hardened would be false.

Classical.
Every set has a classical floor of at least 2^λ. For the Q sets it is about $2^{2\lambda}$.

Both tiers assume the hidden orientation stays ineffective (§8).

7 Attacks

This is every attack we know, with its status. Costs are in Table 8. "Measured" means we ran the attack or its oracle. "Derived" means it was computed from the parameters.

7.1 Key recovery

The secret key is a function of f (measured).
Given f, a working key follows in polynomial time. The Kani scalars come from one sum-of-two-squares, and the eigenvalue scaling is irrelevant. So key recovery means finding f. The test was run on real keys (`eigenx:src/cryptanalysis/degree.rs`).

Degree search with the public-key test (measured; new).
Guess f and test it with a Kani chain of length $e \approx 2\log_2 f$ on $E_A \times E_A$ for $\gamma = u\psi_A + c$, where $\psi_A = 2\tau - tf$ has degree $7f^2$. The test needs no ciphertext and is about half a decapsulation. A wrong guess is rejected only at the final split. Near misses that agree with f on many low bits fare no better, so there is no bit-by-bit recovery. In the 2026-10-17 assessment, 158 wrong candidates across L1, L3 and L5 were all rejected and the true degree was always accepted [Ano26a]. The search space is about $2^{2\lambda+10}$ admissible degrees, so this is not binding.

Generic endomorphism ring or path finding (derived).
ε is with overwhelming probability the unique shortest isogeny from E_D to E_A. So any connecting path yields f after lattice reduction. The cost is $p^{1/2}$ classically, or $p^{1/3+o(1)}$ heuristically with large memory [Wes26], and $p^{1/4}$ quantumly. Since $p \approx 2^{10.5\lambda}$ this is never close.

Meet in the middle on ε (derived).
This needs f to be smooth and its factorisation known. It costs at least $\sqrt{f}$, about $2^{\lambda+10}$.

SIDH torsion attacks on the public key (derived).
The public key gives ε on $E_D[2^E]$ up to a diagonal matrix, with $2^E \gg f^2$. The Castryck–Decru, Maino–Martindale et al. and Robert attacks [CD23, MMP+23, Rob23] need the degree. With f hidden they reduce to the degree search. Hiding the degree is the countermeasure, as in terSIDH and binSIDH [BF23].

Hidden orientation (derived; the main risk).
E_A is oriented by $\mathbb{Z}[f\omega]$, and the public eigenlines are horizontal 2-walks of length E in that orientation. With an effective orientation, the class-group action could be evaluated. Kuperberg's sieve would then recover the key in subexponential quantum time [Wes22], and one endomorphism would give $\mathrm{End}(E_A)$ in $|7f^2|^{1/4}$ classical time [MW23]. No algorithm we know makes it effective (§8).

Self-pairings (derived: not applicable).
These need $m^2 \mid \mathrm{disc}$ for accessible m [CHM+23]. The discriminant $-7f^2$ is odd and prime to 3.

The earlier public-degree variant (measured: broken).
The toy prototype's "walk" variant used a public degree 5^k. Its key is a function of k, and it was broken on 5 of 5 ciphertexts. Version 2 exists to fix this.

7.2 Message recovery

Mask guessing plus a torsion attack (derived; binding).
The pairing reveals the product of the masks but not their ratio. Guessing the ratio modulo

2^n with $2^{2n} > d$ gives the torsion data for Robert's attack. The cost is $\sqrt{d} \approx 2^\lambda$ classically and $d^{1/4}$ Grover queries. This is FESTA's estimate [BMP23] and the binding attack on every set.

Claw finding on the sender's isogeny (derived).
This costs $\sqrt{d}$ classically and $d^{1/3}$ with Tani's algorithm. Tani is no better than classical van Oorschot–Wiener once memory is costed [JS19]. A meet-in-the-middle is implemented in `eigenx:src/cryptanalysis/mitm3.rs`.

Lollipop attacks (derived: not applicable as stated).
The attacks of Castryck and Vercauteren on FESTA and M-SIDH [CV23] need a known low-degree endomorphism with the public basis as eigenbasis. The only such endomorphisms are in $\mathbb{Z}[\tau]$, which has degree at least $2f^2$ and is unknown without f. No element of $\mathbb{Z}[\tau]$ has an eigenline on the 3^F-torsion, because 3 is inert in $\mathbb{Q}(\sqrt{-7})$.

7.3 Chosen-ciphertext, implementation and physical attacks

Adaptive and decryption-failure attacks (derived).
FO with implicit rejection answers every malformed ciphertext with a pseudorandom key. The adaptive FESTA attacks [MO23] need a validation or timing oracle, which constant-time decapsulation removes. No honest decapsulation has failed in any recorded run. That is evidence, not a proof that the correctness error is zero.

Timing (partly measured).
The secret phase of decapsulation was shown instruction-count constant on the earlier Mumford build. It has not been re-measured on the theta build. Key generation is variable time by design.

Side channels beyond timing (open).
Leaking α or f through power or EM reduces the scheme to a subexponential quantum problem. Masks are per ciphertext, so the mask-leakage results [DIKM25] apply as to FESTA. Fault attacks on torsion masking [GLM26] also apply, and the FO check is the only countermeasure.

8 Risks, big unknowns and open questions

Ranked by how much would fall if each went wrong.

R1. *The hidden orientation becomes effective.* The public key is a segment of a horizontal 2-walk in the hidden $\mathbb{Z}[f\omega]$-orientation. Turning that into an evaluable class-group action, for example with the level-structure frameworks [FFP24, A$^+$24] or sesquilinear pairings, gives subexponential quantum key recovery at *every* set. Larger parameters do not help, because the Q sets enlarge d and not f. Nobody has tried. **This is the big unknown.**

R2. *A mask-free higher-dimensional embedding.* The subgroups $\varphi\langle P_A\rangle$ and $\varphi\langle Q_A\rangle$ and the mask product are public. A dimension-4 or -8 Kani embedding from these alone would remove the $\sqrt{d}$ search behind every security level. This is open and not ruled out.

R3. *The eigenbasis gives a handle on f.* The attacker knows τ on $E_A[2^E]$ up to the scalar f. We found no use for this beyond guessing f whole, and the public-key test needs all of f. A partial-information attack would break key recovery.

R4. *The decisional assumption.* IND-CPA needs a decisional problem. Every basis is an eigenbasis of some endomorphism, so the assumption is about the size of the smallest diagonal one. It has no argument beyond the absence of a distinguisher.

R5. *Correctness error.* The FO bound has a term $q_D\delta$. We have observed $\delta = 0$ in every run but have no proof for the fixed chain schedule.

R6. *The Grover-oracle constant.* The gate-count tier rests on an oracle cost model. Its chain-step constant now matches our exact count, rounded down (§6); it was 2^8, which overcharged the attacker by 0.5–1.0 bit in gates. It has had no external review. Tier B does not depend on it.

R7. *Implementation.* Constant-time behaviour has not been re-verified on the theta build. Key generation is variable time. There is no second implementation and no independent check of the known-answer tests.

8.1 Open questions

Q1. Can the two public 2-walks, possibly with level structure or a sesquilinear pairing, make the $\mathbb{Z}[f\omega]$-orientation of E_A effective? (R1)

Q2. Can a Kani embedding of dimension 4 or 8 be built from the masked subgroups and ab alone, without guessing the ratio? (R2)

Q3. Is there a test that rejects a wrong degree using less than all of f, for example modulo 2^j through a smaller endomorphism? (R3)

Q4. Is decisional eigen-CIST well posed, and is there a distinguisher on the size of the smallest diagonal endomorphism? (R4)

Q5. Can the correctness error be proved zero for every admissible key? (R5)

Q6. Engineering: decrypt with two half-chains to bring p from about d^5 to d^3, and add compressed keys and ciphertexts and constant-time key generation.

9 Implementation status

- Rust crate in the sibling repository `eigenx`: constant-time $\mathbb{F}_p$ arithmetic (Montgomery-friendly reduction, safegcd inversion), x-only ladders, 3-isogeny chains, and a theta-coordinate $(2,2)$-chain for decapsulation. It has a CLI with a versioned file format, known-answer tests, hostile-input tests and CI.
- The cost model (`src/security.rs`) and runnable attack components (`src/cryptanalysis/`) are in the same crate.
- A NIST package draft (`docs/nist/`) states what is still missing: an external review of the FO proof, a proof of zero correctness error, constant-time evidence on the current source, a C reference implementation, NIST-DRBG KATs, and timings on the reference platform.

10 Reproduction

- Benchmark: `cargorun--release--examplebench_kem--<set><keys><trials><jobs>` at `eigenx` commit `d89d790`. Records, host description, graphs and the run report are in `crypto:research/eigen_seta_whitepaper_20261017/`, with copies in `crypto:benchmarks/20261017-eigen-seta-whitepaper/`.
- Security levels: `cargorun--release--examplesecurity_estimate`.
- Specification, proofs and the full analysis: the companion paper [Ano26a], `crypto:papers/eigen-seta/`.

References

[A+24] Sarah Arpin et al. Level-structure group actions on supersingular curves. Cryptology ePrint Archive, Paper 2024/1172; cited as in `eigenx:docs/SECURITY.md`, 2024.

[Ano26a] Anonymous. Eigen-séta: an isogeny KEM whose trapdoor is a hidden-degree endomorphism (revision 2). Repository file `crypto:papers/eigen-seta/eigen_seta.pdf`, 2026.

[Ano26b] Anonymous. EigenX: Rust implementation of the Eigen-Séta KEM and PKE, with its security analysis. Sibling repository `eigenx`: `docs/SECURITY.md`, `docs/LITERATURE_DRILL.md`, `src/security.rs`, 2026.

[BBC+21] Gustavo Banegas, Daniel J. Bernstein, Fabio Campos, Tung Chou, Tanja Lange, Michael Meyer, Benjamin Smith, and Jana Sotáková. CTIDH: faster constant-time CSIDH. In *IACR Transactions on Cryptographic Hardware and Embedded Systems*, volume 2021, 2021.

[BF23] Andrea Basso and Tako Boris Fouotsa. New SIDH countermeasures for a more efficient key exchange. In *Advances in Cryptology – ASIACRYPT 2023*, volume 14444 of *LNCS*, pages 208–233. Springer, 2023.

[BM24] Andrea Basso and Luciano Maino. POKÉ: A compact and efficient PKE from higher-dimensional isogenies. EUROCRYPT 2025; Cryptology ePrint Archive, Paper 2024/624, 2024.

[BMP23] Andrea Basso, Luciano Maino, and Giacomo Pope. FESTA: Fast encryption from supersingular torsion attacks. In *Advances in Cryptology – ASIACRYPT 2023*, volume 14444 of *LNCS*, pages 98–126. Springer, 2023.

[CD23] Wouter Castryck and Thomas Decru. An efficient key recovery attack on SIDH. In *Advances in Cryptology – EUROCRYPT 2023*, volume 14008 of *LNCS*, pages 423–447. Springer, 2023.

[CHM+23] Wouter Castryck, Marc Houben, Simon-Philipp Merz, Marzio Mula, Sam van Buuren, and Frederik Vercauteren. Weak instances of class group action based cryptography via self-pairings. In *Advances in Cryptology – CRYPTO 2023*. Springer, 2023. Cryptology ePrint Archive, Paper 2023/549.

[CV23] Wouter Castryck and Frederik Vercauteren. A polynomial time attack on instances of M-SIDH and FESTA. In *Advances in Cryptology – ASIACRYPT 2023*, volume 14444 of *LNCS*, pages 127–156. Springer, 2023. Cryptology ePrint Archive 2023/1433.

[DIKM25] Saikat Das, Riccardo Invernizzi, Péter Kutas, and Jonas Meers. Leveled isogeny problems with hints. PKC 2026; Cryptology ePrint Archive, Paper 2025/2075, 2025.

[FFP24] Luca De Feo, Tako Boris Fouotsa, and Lorenz Panny. Isogeny problems with level structure. Cryptology ePrint Archive, Paper 2024/459, 2024.

[GLM26] Valerie Gilchrist, Yi-Fu Lai, and Michael Meyer. Fault injection attacks on torsion masking. Cryptology ePrint Archive, Paper 2026/1944, 2026.

[JS19] Samuel Jaques and John M. Schanck. Quantum cryptanalysis in the RAM model: Claw-finding attacks on SIKE. In *Advances in Cryptology – CRYPTO 2019*, volume 11692 of *LNCS*, pages 32–61. Springer, 2019.

[MMP+23] Luciano Maino, Chloe Martindale, Lorenz Panny, Giacomo Pope, and Benjamin Wesolowski. A direct key recovery attack on SIDH. In *Advances in Cryptology – EUROCRYPT 2023*, volume 14008 of *LNCS*, pages 448–471. Springer, 2023.

[MO23] Tomoki Moriya and Hiroshi Onuki. Adaptive attacks against FESTA without input validation or constant-time implementation. Cryptology ePrint Archive, Paper 2023/1092, 2023.

[MW23] Arthur Herlédan Le Merdy and Benjamin Wesolowski. The supersingular endomorphism ring problem given one endomorphism. IACR Communications in Cryptology 2(1), 2025; Cryptology ePrint Archive, Paper 2023/1448, 2023.

[PIK26] PIKE: Faster isogeny-based public key encryption with pairing-assisted decryption. Cryptology ePrint Archive, Paper 2026/473 (abstract: NIST I, C on Intel i7 2.3 GHz, 53/34 Mcycles enc/dec), 2026.

[Rob23] Damien Robert. Breaking SIDH in polynomial time. In *Advances in Cryptology – EUROCRYPT 2023*, volume 14008 of *LNCS*, pages 472–503. Springer, 2023.

[Wes22] Benjamin Wesolowski. Orientations and the supersingular endomorphism ring problem. In *Advances in Cryptology – EUROCRYPT 2022*. Springer, 2022. Cryptology ePrint Archive, Paper 2021/1583.

[Wes26] Benjamin Wesolowski. The supersingular isogeny problem in time and memory $p^{1/3+o(1)}$. Cryptology ePrint Archive, Paper 2026/1486, 2026.